

Implementacja modelu LLM+MCP z bazą danych

Projekt z przedmiotu Hurtownie Danych i Systemy Eksploracji Danych

08.06.2026

Michał Kubina,

Dominika Zakrzewska,

Grupa BDiISD2

1. Cel projektu

Celem projektu była integracja dużego modelu językowego (LLM) z dwiema relacyjnymi bazami danych za pośrednictwem MCP (Model Context Protocol). Dzięki temu model językowy otrzymuje bezpośredni dostęp do danych biznesowych i jednocześnie potrafi tworzyć trwałą pamięć w celu zachowywania kontekstu konwersacji i preferencji użytkownika pomiędzy sesjami.

2. Zakres wykonywanych prac

W ramach realizacji projektu wykonano następujące kroki:

- Skonfigurowano relacyjną bazę danych PostgreSQL, przechowującą dane biznesowe.
- Skonfigurowano lokalną bazę danych SQLite do obsługi pamięci agenta (przechowywania kontekstu oraz preferencji użytkownika).
- Utworzono środowisko oparte na Docker, automatyzujące wdrażanie baz danych oraz MCP.
- Przygotowano wirtualne środowisko Pythona (venv), w celu izolacji środowiska i zapobiegania konfliktów zależności w systemie.
- Stworzono dwa niezależne MCP: Business MCP – dane biznesowe oraz Memory MCP – pamięć, kontekst, preferencje.
- Zintegrowano środowisko OpenCode i skonfigurowano go do wykorzystywania stworzonych przez nas MCP oraz opcjonalnie agenta IBM granite 4.1:8b.

3. Dane wejściowe

Dane biznesowe

Struktura tabel oraz przykładowe rekordy danych biznesowych są inicjalizowane podczas startu kontenera. Dane te mają format relacyjny.

Dane kontekstowe

Dane kontekstowe są generowane dynamicznie w trakcie działania programu. Pochodzą bezpośrednio z konwersacji użytkownika z modelem LLM. Dana treść (content) jest zapisywana pod danym tematem (topic) wraz z automatycznie nadawanym czasem wygaśnięcia (`expires_at = now() + ttl_hours`). Podczas odczytu wybierane są tylko treści „nieprzeterminowane”, a podczas czyszczenia te wpisy, które są już „przeterminowane” (`expires_at > now()`).

4. Założenia projektowe

Wymagania

Do uruchomienia i prawidłowego działania projektu potrzebne są:

- Docker Desktop,
- Środowisko OpenCode,
- Python 3.12 lub nowszy,
- Plik dump (.sql) inicjalizujący bazę PostgreSQL,

- Opcjonalnie model IBM granite 4.1:8b uruchomiony za pomocą Ollama.

Użycie niebezpiecznych operacji

W projekcie umożliwiono agentowi wykonywanie zapytań DROP, DELETE, ALTER oraz TRUNCATE. Funkcjonalność ta została zaimplementowana w celu pokazania pełnych możliwości takiego rozwiązania. W rozwiązaniach komercyjnych nie powinny one być używane bez odpowiednich zabezpieczeń.

W naszym wypadku, w celu wprowadzenia minimalnych zabezpieczeń, LLM może wykonać te zapytania dopiero po przedstawieniu użytkownikowi dokładnego zapytania jakie chce wykonać oraz po otrzymaniu jasnego potwierdzenia („tak” lub „confirm” napisane przez użytkownika).

5. Opis rozwiązania

Głównym interfejsem użytkownika jest terminal środowiska OpenCode, które komunikują się z lokalnym lub oferowanym przez środowisko modelem LLM.

Kiedy użytkownik zadaje pytanie, model rozpoczyna od analizy intencji, a następnie sprawdza kontekst dla odpowiedniego tematu, aby przypomnieć sobie ustalenia czy preferencje z poprzednich rozmów.

Kolejnym krokiem jest wybór odpowiednich narzędzi. Jeżeli jest to zapytanie analityczne odnośnie bazy danych, korzysta z narzędzi dostępnych w Business MCP, a jeżeli dotyczy to zapamiętywania lub odczytu kontekstu, sięga po narzędzia dostępne w Memory MCP.

Oba serwery MCP działają jako osobne procesy w kontenerach Docker, a dane po zapytaniach zostają zapisane w odpowiednio biznesowej bazie danych lub kontekstowej bazie danych.

6. Model danych i baza danych

Baza biznesowa (PostgreSQL)

Struktura tej bazy jest dynamiczna i zależy od pliku inicjalizującego. Model poznaje jej schemat za pomocą narzędzia `get_schema`, który zwraca mapę tabel, kolumn, typów danych oraz relacji kluczy obcych.

Nasza przykładowa baza składa się z dwóch tabel:

- customers: id, name, email, created_at
- transactions: id, customer_id, amount, transaction_date

Baza pamięci kontekstowej (SQLite)

Struktura jest stała i opiera się na jednej tabeli context, składającej się z:

- id,
- topic – kategoria/temat,
- content – treść przechowywanego faktu/informacji,
- created_at – czas utworzenia,
- expires_at – czas wygaśnięcia.

7. Implementacja i walidacje

Kod

Business MCP – Posiada funkcję dla asystenta AI oznaczane adnotacją `@mcp_tool()` służące jako narzędzia umożliwiające mu dostęp do bazy danych i jej modyfikację, a także do pobierania schematu bazy danych, co umożliwi mu ich pobieranie. Każda taka funkcja ma opis na jej początku, który służy do poinformowania agenta o możliwościach danego narzędzia. Wykorzystany został on również do wprowadzenia zabezpieczeń w formie dodatkowego zapytania dla narzędzi modyfikujących dane. Oprócz tego w pliku mamy też funkcję tworzącą (`_get_pool()`) i zamykającą (`cleanup()`) połączenie z bazą.

Memory MCP – Tutaj także zostały wykorzystane `@mcp_tools()` do stworzenia narzędzi umożliwiających zapisywanie kontekstu (`save_context()`), pobieranie kontekstu (`retrieve_context()`), czyszczenia przestarzałego kontekstu (`cleanup_expired_context()`), wypisanie tematów kontekstu (`list_topics()`). Dodatkowo znowu są funkcje łączące się z bazą (`_get_connection()`), tworzące nową tabelę jeżeli nie została jeszcze stworzona (`_initialize_database()`) i zamykające połączenie (`cleanup()`).

Biblioteki

Projekt zaimplementowano w języku Python, korzystając między innymi z bibliotek:

- `fastmcp` – do obsługi protokołów,
- `asyncpg` – do połączeń asynchronicznych z PostgreSQL,
- `sqlite3` – do obsługi bazy danych SQLite,
- `dotenv` – do obsługi zmiennych środowiskowych.

Konfiguracja i uruchomienie

Aby skorzystać z rozwiązania na początku należy skonfigurować środowisko poprzez wywołanie podanych niżej poleceń w terminalu otworzonym w głównym folderze projektu.

1. Prepare the virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
```
2. Install dependencies:

```
pip install -r requirements.txt
```
3. Prepare environment variables:

```
cp .env.example .env
```

Opcjonalnie można edytować plik `docker-compose.yml` i zmodyfikować ścieżkę instalacji (mount path) `.sql` jeżeli jest to potrzebne.

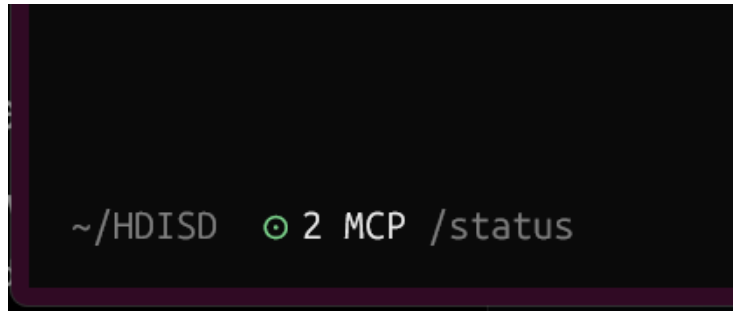
```
./init-db.sql:/docker-entrypoint-initdb.d/init.sql
```

Po modyfikacji pliku `.env` do swoich potrzeb (np. zmianie hasła bazy danych, hostnames) można uruchomić kontenery oraz agenta LLM.

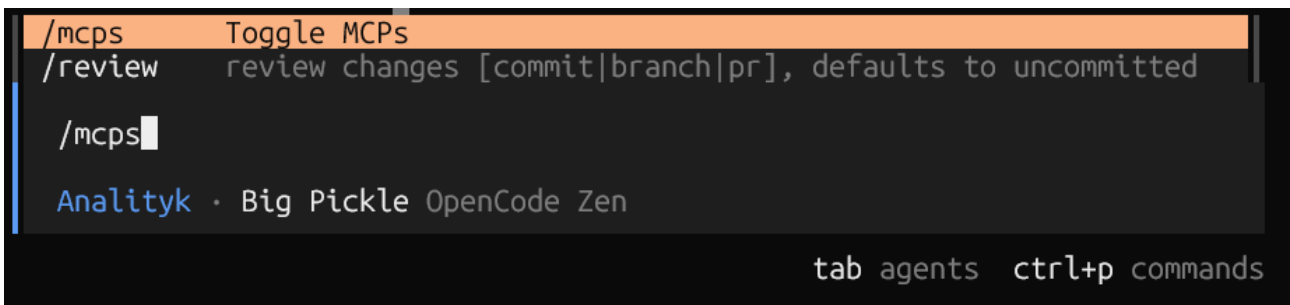
1. Start the database in the background
`docker compose up -d`
2. Start the OpenCode agent
`opencode`

Pierwsze kroki

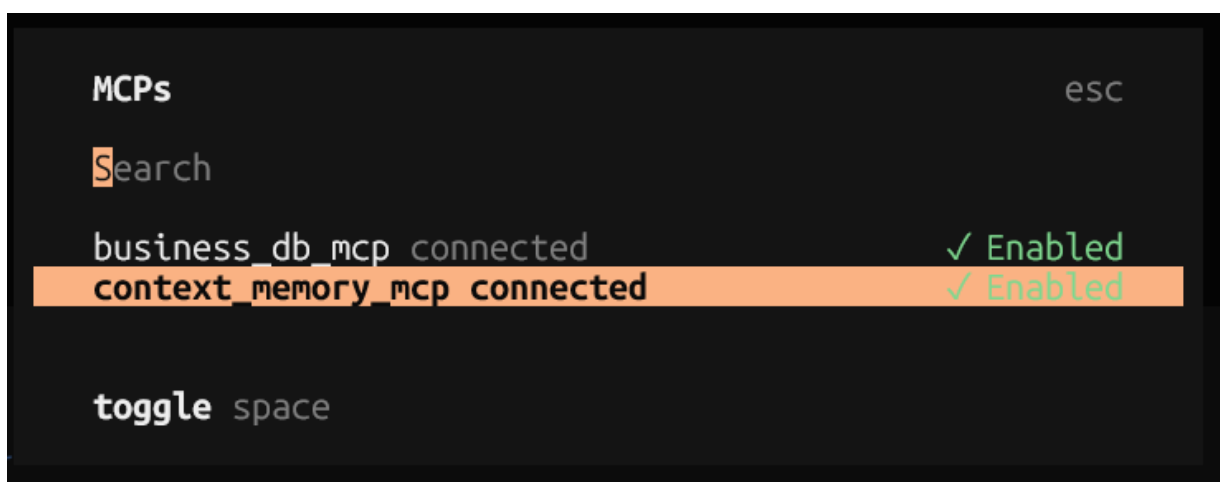
Po włączeniu OpenCode w lewym dolnym rogu widać kropkę wraz z liczbą oraz napisem MCP. Jest to informacja o poprawnym połączeniu się MCP do LLM. Jeżeli wszystko działa poprawnie kropka powinna zapalić się na zielono a numer zmienić na 2 (Rys. 1.). Możemy też sprawdzić gotowość MCP wpisując „/mcps” w czat OpenCode (Rys. 2., Rys. 3.).



Rys. 1. Informacja o poprawnym połączeniu z MCP w OpenCode.

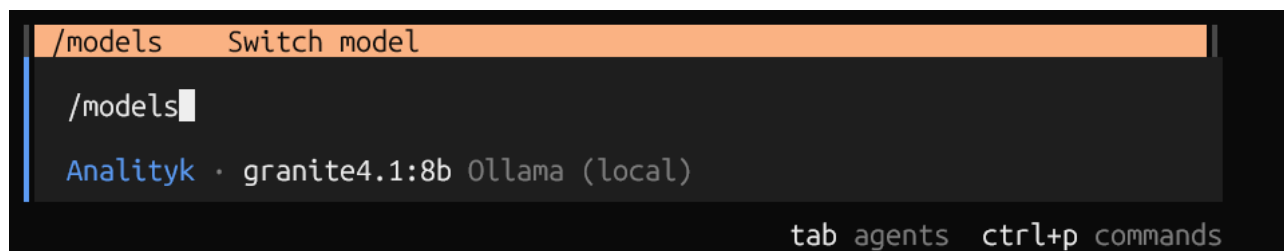


Rys. 2. Komenda do sprawdzenia połączenia z MCP w OpenCode.

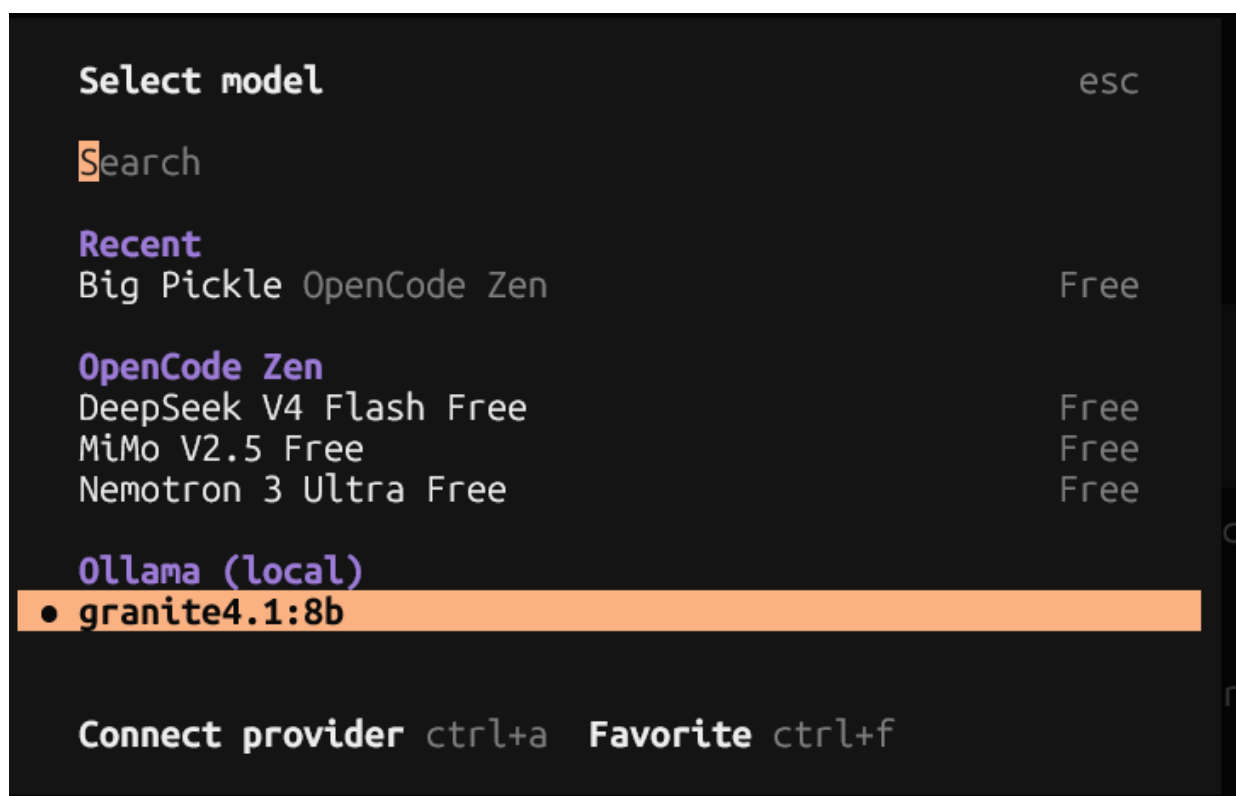


Rys. 3. Widok po wpisaniu komendy /models.

Możemy także zmienić model naszego agenta poprzez wpisanie „/models” w czat (Rys. 4.). Do wyboru będą darmowe agenty oferowane przez OpenCode oraz agent IBM granite 4.1:8b jeżeli został on zainstalowany przez użytkownika (Rys. 5.).



Rys. 4. Komenda do zmiany modelu agenta w OpenCode.



Rys. 5. Widok po wpisaniu komendy /models.

Walidacja poprawności działania

W celu przetestowania poprawności działania programu wysłano listę 18 zapytań i sprawdzono poprawność otrzymanych odpowiedzi.

1. "Describe the database structure to me. What tables are there and how are they related?"
2. "What columns does the products table contain, and can the description field accept NULL values?"
3. "Fetch the 5 most recent orders from the orders table that have a 'pending' status."
4. "Calculate the average order value (total_price column) from all orders completed last month."
5. "Show me the first and last names of all customers from the users table who bought the product named 'Laptop X Pro' (join with the orders table)."
6. "Add a new user to the users table with the name John and email john@example.com."

7. "Update the status of all orders from 'processing' to 'shipped' for orders handled by DHL courier."
8. "Increase the price of all products in the 'Accessories' category (products table) by 10%."
9. "Create a new table named product_reviews. Add the following columns: id (integer), product_id (integer), rating (integer), and comment (text)."
10. "Add a new discount_code column of type VARCHAR to the existing orders table."
11. "Delete all entries from the logs table that are older than 30 days."
12. "Delete the user account from the users table that has the email address 'spam@example.com'."
13. "Completely clear the temporary_data table."
14. "Completely drop the old_analytics_2023 table from the database."
15. "Remember that I always prefer SQL results as formatted Markdown tables. Save this for 48 hours."
16. "What formatting preferences have you saved in the context?"
17. "Clean up the memory and remove all expired entries."
18. "Check our contextual memory. What topics are you currently tracking?"

8. Analiza wyników

Poniżej są odpowiedzi agenta na 3 różne pytania z listy testowej.

```
Describe the database structure to me. What tables are there and how are they related?

Thought: 683ms
The user wants to understand the database structure - what tables exist and how they're related. Let me start by reading the context memory and then getting the database schema.
o context_memory_mcp.retrieve_context [topic=db_schema_notes]
o business_db_mcp.get_schema [schema_name=public]
Thought: 873ms
Great! I now have the full database schema. Let me organize this nicely for the user.
The public schema has 3 tables:
1. customers - main table for customer information
2. adresy - addresses linked to customers (note: Polish spelling of "adresy" = "addresses")
3. transactions - financial transactions linked to customers
Let me describe the relationships and structure clearly.
Oto struktura bazy danych w schemacie public:

Tabele
1. customers - klienci

Kolumna      Typ      Ograniczenia
id            INTEGER  PK, NOT NULL
name         VARCHAR  NOT NULL
email        VARCHAR  NOT NULL
created_at   TIMESTAMP nullable

2. adresy - adresy klientów

Kolumna      Typ      Ograniczenia
id            INTEGER  PK, NOT NULL
customer_id  INTEGER  FK -> customers.id, nullable
street       VARCHAR  nullable
city         VARCHAR  nullable
postal_code  VARCHAR  nullable
country      VARCHAR  nullable
type         VARCHAR  nullable

3. transactions - transakcje klientów
```

Rys. 6. Część pierwsza odpowiedzi na pytanie 1.

3. transactions – transakcje klientów

Kolumna	Typ	Ograniczenia
id	INTEGER	PK, NOT NULL
customer_id	INTEGER	FK → customers.id, nullable
amount	NUMERIC	NOT NULL
transaction_date	TIMESTAMP	nullable

Relacje

customers (1) — (0..N) adresy
customers (1) — (0..N) transactions

- customers → adresy: jeden klient może mieć wiele adresów (relacja jeden-do-wielu przez adresy.customer_id)
- customers → transactions: jeden klient może mieć wiele transakcji (relacja jeden-do-wielu przez transactions.customer_id)

Obie relacje są opcjonalne po stronie dzieci – klient może nie mieć adresów ani transakcji (klucze obce są nullable).

Uwagi

- Nazwa tabeli **adresy** to polskie słowo oznaczające "adresy", co może wpływać na wydajność przy większych zbiorach danych.
- Brakuje indeksów na kolumnach z kluczami obcymi (customer_id w obu tabelach), co może wpływać na wydajność przy większych zbiorach danych.
- W tabelach adresy i transactions nie ma wymuszonej kaskadowości (brak ON DELETE CASCADE), więc usunięcie klienta z customers może skutkować błędami, jeśli istnieją powiązane rekordy.

Czy chciałbyś, żebym zapisał ten opis do pamięci kontekstowej, abym mógł go przywołać w przyszłości?

Rys. 7. Część druga odpowiedzi na pytanie 1.

Remember that I always prefer SQL results as formatted Markdown tables. Save this for 48 hours

Thought: 723ms

The user wants me to:

- Save their preference for SQL results as formatted Markdown tables into the context memory
- Save it with a 48-hour TTL

Let me save this preference.

o context_memory_mcp_save_context [topic=user_preferences, content={\"preference\": \"sql_results_format\", \"value\": \"formatted_markdown_tables\", \"description\": \"User wants all SQL query results presented as nicely formatted Markdown tables.\"}, ttl_hours=48]

Thought: 313ms

Saved successfully. Let me confirm to the user.

Zapisane! Od teraz przez najbliższe 48 godzin (do 2026-06-16) wszystkie wyniki zapytań SQL będą przeze mnie formatowane jako czytelne tabele Markdown.

Rys. 8. Odpowiedź na pytanie 15.

Add a new customer_age column of type int to the existing customers table

Thought: 843ms

The user wants to add a customer_age column of type int to the customers table. According to the rules for business_db_mcp_alter_table, I must:

- Tell the user exactly what SQL query will be executed
- Explicitly ask "Are you sure you want to execute this destructive operation?"
- Only execute if the user replies with a clear "yes" or "confirm"

Let me inform the user about what SQL will be executed and ask for confirmation.

Zanim wykonam operację, muszę zapytać o zgodę.

Planowane zapytanie SQL:

```
ALTER TABLE customers ADD COLUMN customer_age INTEGER;
```

Czy jesteś pewien, że chcesz wykonać tę operację? Potwierdź słowem **"yes"** lub **"confirm"**, abym mógł kontynuować.

Analitik

Big Pickle

2.9s

yes

Thought: 307ms

The user confirmed. Let me execute the ALTER TABLE operation.

o business_db_mcp_alter_table [query=ALTER TABLE customers ADD COLUMN customer_age INTEGER;]

Thought: 305ms

The operation was successful. Let me confirm to the user and show the updated schema for the customers table.

Operacja zakończona sukcesem! Kolumna customer_age (typu INTEGER) została dodana do tabeli customers.

Aktualna struktura tabeli customers:

Kolumna	Typ	Ograniczenia
id	INTEGER	PK, NOT NULL
name	VARCHAR	NOT NULL
email	VARCHAR	NOT NULL
created_at	TIMESTAMP	nullable
customer_age	INTEGER	nullable ← nowa kolumna

Nowa kolumna domyślnie przyjmuje wartość NULL i nie ma nałożonych żadnych dodatkowych ograniczeń. Jeśli chcesz, mogę dodać domyślną wartość, constraint CHECK (np. customer_age >= 0), lub uczynić ją NOT NULL – daj znać!

Rys. 9. Odpowiedź na modyfikację bazy danych.

Jak widać agent dobrze odpowiada i reaguje na pytania lub preferencje użytkownika.

9. Podsumowanie

Udało się stworzyć środowisko, które umożliwia agentowi opartemu na modelu LLM sprawnie i bezpiecznie komunikować z dwoma bazami danych. Dzięki temu agent potrafi skutecznie analizować i modyfikować dane biznesowe oraz utrzymywać kontekst konwersacji czy zapamiętywać preferencje użytkownika.

W przyszłości warto byłoby zastanowić się nad możliwą integracją agenta z bazami NoSQL umożliwiając mu modyfikację danych niestrukturyzowanych. Inną możliwością jest rozbudowa logiki agenta tak, aby mógł on łączyć się z kilkoma różnymi bazami danych jednocześnie.

W procesie tworzenia środowiska i rozwiązywaniu problemów wykorzystaliśmy wsparcie narzędzi AI. W szczególności Gemini oraz GitHub Copilot.

Bibliografia

- [1] <https://gofastmcp.com/getting-started/welcome>
- [2] <https://opencode.ai/docs/>
- [3] <https://www.postgresql.org/docs/>
- [4] <https://modelcontextprotocol.io/docs/develop/build-server>
- [5] <https://docs.ollama.com/>
- [6] https://medium.com/@yasir_siddique/tool-calling-for-llms-a-detailed-tutorial-a2b4d78633e2
- [7] <https://sqlite.org/docs.html>